Lab 3: Architectural Pattern Selection – Written Justification

Nishkal M R

PES1UG24AM181

Assigned scope: Mentor matching and session booking (FR-001 to FR-004, NFR-002).

We chose Microservices Architecture for the assigned subsystem of the Mock Interview and Alumni Mentorship Platform.

Style	Pros	Cons	Fit for this scope
Layered	Simple, clear separation	Scales as one block; tight coupling	Weak: cannot scale booking alone
Client-Server	Central control, easy consistency	Single point of failure, bottleneck	Weak: peak-load bottleneck
Microservices	Independent scaling, fault isolation	Operational complexity, network latency	Best: booking scales; calendar failure isolated

Architectural Choice

Microservices Architecture.

Two Reasons

1. Booking is the busiest and most critical operation. Students compete for the same mentor slots at peak times, so the Booking Manager must scale on its own, without scaling the matching or calendar code with it.
2. Calendar and meeting-link creation depends on an external Calendar Service that can be slow or fail. As a separate service, a calendar failure is isolated and does not stop students from browsing mentors, and booking can retry the sync later.

Security Advantage

Each service owns a narrow API and runs in its own process. Slot data is reachable only through the Booking Manager and Matching Service via authenticated REST calls, so a compromise of one service (for example, the UI-facing matching service) does not expose the whole system, and access rules can be enforced at every service boundary.

Performance Benefit

Booking Manager instances can be replicated independently under peak load, and slot reservation uses a single database transaction (check and reserve), which prevents double-booking while keeping response time within the NFR-002 target.